

Scripting

Script

Un *script* es un archivo que contiene una sucesión de comandos de Shell que pueden ejecutar diversas tareas de acuerdo al contenido del texto del *script*. Un *script* nos permite automatizar acciones que bien pueden ser repetitivas o simplemente nos permiten administrar sistemas operativos de una forma más eficiente.

Para que funcione el *script* debe escribirse con una lógica particular, ya que Bash ejecuta el código en el orden en que se escribe.

Para que Linux entienda la diferencia entre un archivo común, de un archivo que convertiremos en *script*, debemos usar los 2 primeros bytes del mismo para clasificarlo.

A diferencia de otros sistemas operativos en los que la extensión define el tipo, en Linux es indistinto. Por convención, agregaremos al nombre del archivo la extensión “.sh” con el fin de que a simple vista podamos identificarlo, pero tengan presente que el sistema operativo no lo necesita. En el caso de los *scripts*, los 2 primeros bytes se denominan “SHE-BANG” o “SHA-BANG” y se le agrega “#!”, seguido del intérprete.

Ejemplo

```
#!/bin/bash
```

Para ejecutar un *script*, el archivo debe tener permisos de ejecución para el usuario o el grupo. Supongamos que tenemos un *script* llamado “control.sh”, este archivo debe tener permisos al menos 755 (rwx=owner, r-x=group, r-w=other), para que el dueño los pueda leer, modifica y ejecutar. Si fuera necesario, miembro del grupo u otros, podrían leer y ejecutarlo.

Lo siguiente que debemos tener en cuenta para ejecutarlo, es conocer su ubicación. Si sabemos que “control.sh” está en el directorio “/home/alumno/”, podríamos invocarlo escribiendo la ruta completa incluyendo el nombre del *script*:

Ejemplo

```
"/home/alumno/control.sh"
```

Otro método para ejecutarlo, es introducir el *script* en el PATH declarado en las variables de entorno¹ (ej.: /usr/local/bin). Técnicamente funcionaría como el ejemplo anterior, salvo que no haría falta poner el PATH completo, bastaría con llamar al *script* sólo por su nombre.

Ejemplo

```
control.sh
```

También podríamos invocar al intérprete con el nombre del *script*:

```
bash control.sh
```

O directamente de esta forma, si estamos dentro del directorio donde está ubicado el *script*:

```
./control.sh
```

Variables

Una variable es un espacio reservado en memoria que se utiliza para almacenar datos o valores. En BASH, las variables son dinámicas, lo que significa que no es necesario declarar su tipo antes de usarlas; simplemente las definimos y asignamos un valor.

En BASH podemos encontrar diferentes tipos variables:

Variables creadas por el programador

Son definidas por el usuario dentro de un *script* o en la terminal al momento de usarlas. Es de buena práctica nombrar a las variables totalmente en mayúsculas.

Ejemplo

Para definirla:

```
SALUDO=Bienvenido
```

Para invocarla, utilizaremos el comando “echo” el cual imprime por pantalla el contenido de la variable al que le deberemos anteceder el signo “\$”:

¹ **Nota:** para conocer cuáles son los PATH de las variables de entorno se debe ejecutar “\$PATH”

```
echo $SALUDO
```

```
Bienvenido
```

```
root@debian11-CA:~# SALUDO=Bienvenido  
root@debian11-CA:~# echo $SALUDO  
Bienvenido
```

Si dentro de la variable lo que buscamos es ejecutar un comando, el contenido de la variable deberá ser encerrado con el acento invertido “`”.

Ejemplo

```
root@debian11-CA:~# LISTAR=`ls`  
root@debian11-CA:~# echo $LISTAR  
archivo hl_archivo messages sl_archivo.txt
```

Variables intrínsecas (predefinidas)

Estas son variables que BASH define automáticamente, y que proporcionan información del entorno o controlan el comportamiento del Shell; estas variables estarán siempre disponibles y no necesitan ser definidas.

Ejemplo

- \$HOME: ruta del directorio del usuario.
- \$USER: nombre del usuario actual.
- \$PATH: directorios donde el sistema busca comandos ejecutables.
- \$PWD: directorio de trabajo del usuario.

Variables especiales

Predefinidas por BASH, permiten acceder a información del *script* y su ejecución.

- \$0: nombre del *script*.
- \$1...\$n: variables que almacenan los argumentos opcionales que podrían proporcionarse al momento de ejecutar el *script*.
- \$#: variable que contiene el total de los argumentos proporcionados.

- `$*`: conjunto de los argumentos.
- `$?`: valor de ejecución del comando anterior (*return code*).
- `$$`: identifica el PID del *script*.
- `#!`: identifica el último proceso arrancado en *background*.

Caracteres especiales

Existen caracteres que tienen en sí su propio uso dentro de los *scripts* que es necesario conocer.

- `\`: la barra invertida le indica a BASH que ignore el carácter especial que viene después.
- `" "`: las comillas dobles que encierra un texto o una variable se las toma como una cadena entera.

```
SALUDO=Bienvenido Alumno (son 2 cadenas diferentes)
```

```
SALUDO="Bienvenido Alumnos" (1 sola cadena)
```

- `` ``: el acento hacia la izquierda se utiliza para convertir la salida de un comando en el contenido de una variable.
- `' '`: las comillas simples se usa para desactivar todos los caracteres especiales encerrados dentro de ellas.
- `$`: identifica lo que le sigue como una variable.
- `#`: el numeral desactiva el texto que le continúa, se utiliza para generar comentarios.
- `;`: el punto y coma se usa para separar la ejecución de distintos comandos en una misma línea.

Estructuras de control en BASH

Las estructuras de control permiten que un *script* tome decisiones basadas en condiciones particulares que se evalúan de acuerdo a comparaciones que proponemos. Permiten manejar el flujo de ejecución de un *script* de manera dinámica.

IF (Condicional)

Evalúa una condición y ejecuta un bloque de comandos si esta es verdadera. Básicamente, permite "tomar decisiones".

Ejemplo

Evalúo existencia de archivo.

```
if [ -f archivo.txt ]; then
    echo " El archivo existe"
fi
```

La estructura condicional tiene 5 palabras claves, aunque las tres utilizadas en el ejemplo son las imprescindibles para una condición:

- 1) **if** = si
- 2) **elif** = también si
- 3) **else** = de cualquier otra manera
- 4) **then** = entonces
- 5) **fi** = *if* invertido, indica cierre

Ejemplo

Comparamos cadenas de texto.

```
#!/bin/bash
VAR1=HOLA
VAR2=CHAU
if [[ VAR1 == VAR2 ]]; then
    echo "son iguales"
else
    echo "son distintas"
fi
```

WHILE (Bucle Condicional)

Ejecuta un bloque de comandos **mientras** una condición sea verdadera. La explicación rápida sería. Si **while** fuera un semáforo, “sigue mientras la luz sea verde”.

Ejemplo

Creo un contador y muestro el resultado, mientras éste sea menor a 5.

```
#!/bin/bash
CONTADOR=0
while [[ $CONTADOR -lt 5 ]];do
    echo "Contador: $CONTADOR"
    CONTADOR=$((CONTADOR + 1))
done
```

Esta estructura está compuesta por tres palabras:

while = mientras

do = hacer

done = hecho

Ejemplo

Mientras el número ingresado sea distinto a 0, se pedirá ingresar otro número.

```
#!/bin/bash
while [[ $VAR != 0 ]];do
    echo "El numero ingresado es: $VAR"
    read -p "ingrese un numero: " VAR
done
echo " Como el numero que se ingreso es $VAR, ya puedo salir del WHILE"
```

UNTIL (Bucle Condicional-Inverso)

Ejecuta un bloque de comandos **hasta que** la condición sea verdadera. Es lo opuesto al *while* indicando que se detendrá cuando la condición se cumpla.

Ejemplo

Creo un contador y muestro el resultado hasta que éste sea mayor a 25.

```
#!/bin/bash
CONTADOR=0
until [[ $contador -ge 25 ]]; do
```

```
echo "Contador: $contador"  
contador=$((contador + 1))  
done
```

Operandos en condicionales y bucles

En **BASH**, los operandos y operadores de comparación permiten evaluar condiciones en estructuras condicionales.

Operadores para comparación numérica

- eq**: igual a
- ne**: no igual a
- gt**: mayor que
- ge**: mayor o igual a
- lt**: menor que
- le**: menor o igual a

Operadora para comparación de cadenas

- = o ==**: las cadenas son iguales
- !=**: las cadenas son diferentes
- z**: la longitud de la cadena es cero
- n**: la longitud de la cadena no es cero

Operadores para archivos y directorios

- e**: el archivo o directorio existe
- f**: es un archivo regular
- d**: es un directorio
- r**: el archivo tiene permisos de lectura
- w**: el archivo tiene permisos de escritura
- x**: el archivo tiene permisos de ejecución

Operadora para comparación aritmética

La condición se expresa dentro de `[ [ ] ]` (Doble corchete de apertura y cierre)

`>`: mayor que

`<`: menor que

`>=`: mayor o igual

`<=`: menor o igual

Diferencia entre `[ ]` y `[ [ ] ]` (corchete simple y corchete doble)

La diferencia radica en que el corchete simple es más antiguo y menos flexible, mientras que el corchete doble introdujo mejoras. En el corchete simple se **debe** usar `-eq`, `-lt`, `-d`, etc., para las comparaciones, mientras que al usar corchetes dobles, **además de los anteriores**, permite usar los operadores de comparación aritmética `<`, `>`, etc.

CASE (condicional por coincidencia)

CASE evalúa una variable o cadena contra múltiples patrones y ejecuta el bloque de comandos correspondiente. Útil para manejar condiciones de forma más clara que `if-elif`.

Ejemplo

Clasificar que debo hacer en días de la semana.

```
#!/bin/bash
read -p "Elija un día de la semana" DIA
case $DIA in
    lunes | martes | miercoles ) echo "Ese día tenes que cursar Algebra" ;;
    jueves | viernes ) echo "Ese día tenes que cursar Computacion Aplicada" ;;
    *) echo "Ese día no tenes que cursar peros seria bueno que estudies" ;;
esac
```

También podemos empezar a combinar estructuras entre sí como en el siguiente ejemplo.

Ejemplo

Clasificar que debo hacer en la semana + pregunta recursiva.


```
#!/bin/bash
#read -p "Elija un dia de la semana: " DIA
while [[ $DIA != 0 ]];do
    case $DIA in
        lunes | martes | miercoles ) echo "Ese dia tenes que cursar Algebra" ;;
        jueves | viernes ) echo "Ese dia tenes que cursar Computacion Aplicada" ;;
        *) echo "Ese dia no tenes que cursar, pero seria bueno que estudies" ;;
        0) echo "SALIR"
    esac
    read -p "Elija un dia de la semana: " DIA
done
```

FOR (bucle sobre listas)

En este caso, el **FOR** se utiliza para recorrer una lista iterando sobre los elementos o un rango.

Ejemplo

Recorra y muestra una lista de números.

```
for i in 1 2 3 4 5 6; do
    echo "Numero: $i"
done
```

Ejemplo

Recorra y muestra los valores en un rango.

```
for i in {1..5}; do
    echo "Numero: $i"
done
```

Ejemplo

Recorra y muestra una lista de números de 2 en 2.

```
for i in {1..10..2}; do
    echo "Numero: $i"
done
```

Ejemplo

Recorra la salida de un comando.

```
for i in $(ls -l /var/log/*.log); do
    echo "Archivos con extension log: $i"
done
```

Vectores (o Arrays)

En BASH, los vectores son colecciones de datos indexados que permiten almacenar y acceder a múltiples valores usando un índice. Estos índices están numerados a partir de 0.

Podemos definir un vector con valores dentro.

```
PERSONAJES=("Luke Skywalker" "Han Solo" "Obi-Wan Kenobi")
```

Y obtener uno de esos valores específicos.

```
echo "El personaje elegido es: ${PERSONAJES[0]}"
```

Esto devolver como resultado:

```
El personaje elegido es: Luke Skywalker
```

Los vectores pueden ser recorridos con BUCLES como para no tener que ser llamarlos específicamente por su índice. En este caso, el "@", indica que lo que se va a mostrar es el contenido completo del vector. El vector también puede ser recorrido con "*" en lugar de la arroba.

Ejemplo

\$@

```
for i in "${PERSONAJES[@]}"; do
    echo "El personaje elegido es: $i"
done
```

Ejemplo

\$*

```
for i in "${PERSONAJES[*]}"; do
    echo "El personaje elegido es: $i"
done
```

Nota: recomendamos la creación de estos *scripts* en la máquina virtual para evaluar los resultados de los ejemplos.

Funciones (bloques reutilizables)

Las funciones agrupan comandos en un bloque que puede ejecutarse múltiples veces invocando su nombre.

Ejemplo

Crear un *script* llamado `funcion.sh` que contenga una función que reciba el nombre por argumento.

```
NOMBRE=$1
saludar () {
    echo "Hola, $NOMBRE"
}
saludar
```

Este *script* tiene una función llamada "saludar" que es invocada en la última línea. Cuando se ejecuta el *script* `./funcion.sh "pepe"`; guardara en la variable "NOMBRE" el argumento "pepe" e irá directamente al llamado de la función, cuando esta se ejecute dará el mensaje del "echo" acompañado de la variable.

El siguiente *script* da un ejemplo combinado de lo explicado hasta en este apunte y su implementación.

Ejemplo

Hacer un *script* que combine el uso de bucles y condicionales.

```

#!/bin/bash

PERSONAJES=("Luke Skywalker" "Han Solo" "Leia Organa")

# =====Funciones Reutilizables =====

mostrar_personajes (){
    for i in "${PERSONAJES[@]}";do
        echo " * $i"
    done
}

echo " "

agregar_personaje (){
    read -p "Escribe el nombre del personaje que quieres agregar: "
    PERSONAJE_NUEVO
    PERSONAJES+=("$PERSONAJE_NUEVO")
    echo "Personaje agregado"
}

echo " "

buscar_personaje (){
    read -p "Escribe el nombre del personaje que deseas buscar: "
    PERSONAJE_BUSCAR
    for i in "${PERSONAJES[@]}"; do
        if [[ "$i" == "$PERSONAJE_BUSCAR" ]];then
            echo "Personaje encontrado: $i"
            RESULTADO=1
        fi
    done
    if [[ "$RESULTADO" -eq 0 ]]; then
        echo "El personaje no fue encontrado"
    fi
}

```

```
# =====Seccion de While y MENU =====
while [ OPCION != 0 ];do
    echo "===== "
    echo "===== Menu ===== "
    echo " "
    echo "===== 1) Listar Personajes ===== "
    echo "===== 2) Agregar Personajes ===== "
    echo "===== 3) Buscar Personajes ===== "
    echo " "
    echo "===== "
    read -p "Elija una opcion: " OPCION
    case "$OPCION" in
        1) echo "Listar Personajes"
            mostrar_personajes ;;
        2) echo "Agregar Personajes"
            agregar_personaje ;;
        3) echo "Buscar Personajes"
            buscar_personaje ;;
        0) echo "Salir"
            break
    esac
done
```

Bibliografía

- Niklas, P. (2020). *GNU/Linux, con sabor a Debian*. Pp. 161 – 181.